

Real-Time Android Video Processing Engine

A Native C++ Implementation for Cinematic LUT Rendering

Dipesh Kant(2025MCS2110)
Kunal Dhyani (2025MCS2112)

March 20, 2026

Abstract

Processing high-definition video in real-time requires a massive amount of computational power, which is challenging on mobile devices due to battery and thermal limits. In this group project, we built a custom Android application that captures live camera feeds and applies cinematic 3D color filters (LUTs) in real-time. To achieve high frame rates without overheating the device, we bypassed standard Kotlin code and built a native C++ engine. Through careful memory management, a custom Thread Pool, and ARM NEON SIMD vectorization, we successfully reduced CPU usage from 94% down to 10% while doubling the frame rate. This report details our architecture, mathematical models, results, and our future plans to utilize the GPU via OpenCL.

1 Introduction

Modern smartphones boast powerful cameras, but applying complex color grading (like professional Black and White or Teal & Orange filters) to live video is difficult. Video is made of millions of pixels, and processing them one by one in Kotlin causes severe lag.

Our goal for this project was to build a video processing engine that could apply 3D Lookup Tables (LUTs) to a live camera feed at 30 Frames Per Second (FPS). To do this, our team utilized the Android Native Development Kit (NDK) to write the core mathematical engine in C++, bringing desktop-level performance to a mobile phone.

2 App Structure and Technologies

Our application is built on a hybrid architecture, separating the user interface from the heavy mathematics.

2.1 User Interface and Runtime Configuration

The application features an interactive control overlay that allows for real-time benchmarking and visual adjustment. The menu system (`res/menu/main_menu.xml`) provides the following toggleable parameters:

- **Computational Mode:** Users can switch between the **Baseline (Scalar)** engine, **SIMD (NEON)** engine and **GPU (OpenCL)**. This allows for a direct "side-by-side" performance comparison of hardware-level optimization.
- **Dynamic Resolution Scaling:** The engine supports hot-swapping between **720p (HD)** and **1080p (FHD)**. This functionality tests the memory bandwidth limits of the device's system bus.
- **LUT Selection:** The interface includes a 3D LUT loader with three presets:

- *None*: Bypasses the interpolation math for a pure "Passthrough" benchmark.
- *Teal & Orange*: A cinematic color-grading filter.
- *Black & White*: A high-contrast desaturation filter (as shown in Figures 1 and 2).
- **Live Telemetry Dashboard**: A persistent HUD (Heads-Up Display) monitors the following metrics at 1-second intervals:
 - **LAT**: Frame processing latency in milliseconds.
 - **FPS**: Real-time frames per second rendered.
 - **CPU/MEM**: Processor utilization percentage and PSS memory footprint.
 - **THERMAL**: Battery temperature in Celsius to monitor for hardware throttling.

2.2 The Presentation Layer (Kotlin)

We used Android's modern **CameraX API** within our Kotlin frontend to capture the live video feed. We also designed an interactive user interface that allows the user to switch between different resolutions (720p and 1080p), select different processing modes, and view live telemetry (FPS, CPU usage, Memory, and Temperature).

2.3 Zero-Copy Memory Bridge

Normally, passing huge amounts of video data between Kotlin and C++ causes lag because the device has to copy the data. We solved this by using `ByteBuffer.allocateDirect()`. This allows both the Kotlin camera code and the C++ engine to look at the exact same physical memory block, completely eliminating copy time via the Java Native Interface (JNI).

2.4 The Native C++ Engine

The core processing is handled entirely in C++17. We built a system that takes the raw pixel data, applies complex color math to it, and hands it back to Android to be drawn on the screen.

3 Methodology and Optimization

Our development followed an iterative optimization path, moving from a functional scalar baseline to a hardware-accelerated pipeline.

3.1 Architectural Optimizations

To eliminate common mobile bottlenecks, we implemented the following system-level strategies:

- **Zero-Copy Memory Mapping**: We utilized `ByteBuffer.allocateDirect()` to share raw pixel data between Kotlin and C++. This allows the NDK to access the camera's memory address directly, bypassing the expensive JNI array-copying overhead.
- **Persistent Thread Pool**: Instead of spawning threads per frame (which adds ~5ms of OS latency), we implemented a *Singleton Thread Pool*. Workers stay in a "wait" state using `std::condition_variable` and wake instantly when a new frame is dispatched.

3.2 Hardware-Level Acceleration

We targeted the specific architecture of the Snapdragon 675 (Redmi Note 7 Pro) using two primary techniques:

1. ARM NEON SIMD Vectorization

Standard C++ processes pixels one-by-one (Scalar). We rewrote the inner loops using `**NEON Intrinsics**` to process 128 bits of data per instruction.

- **Parallel Loading:** We load 4 pixels (16 bytes) into a single register.
- **Vector Math:** Using `vmlaq_f32` (Fused Multiply-Add), we perform the Trilinear Interpolation math for all 4 pixels simultaneously in a single CPU clock cycle.

2. Big.LITTLE Thermal Management

Mobile CPUs throttle when "LITTLE" (power-saving) cores struggle with heavy math.

- **The Strategy:** We mathematically capped the worker count to $Total_Cores/2$.
- **The Impact:** This forces the Android Linux kernel to schedule our tasks only on the high-performance "Big" cores. This prevents the "LITTLE" cores from overheating the chip, keeping the CPU at a cool 10% utilization.

4 Mathematical Model: Trilinear Interpolation

A 3D LUT is a digital color cube that maps an input color to a new output color. Because the cube is made of a discrete grid (like $64 \times 64 \times 64$), a pixel's color rarely lands perfectly on a grid point. We use **Trilinear Interpolation** to calculate the final color by mixing the 8 closest colors in the cube.

First, we normalize the pixel's RGB values:

$$\begin{aligned} lut_X &= \left(\frac{R}{255} \right) \times index_{max} \\ lut_Y &= \left(\frac{G}{255} \right) \times index_{max} \\ lut_Z &= \left(\frac{B}{255} \right) \times index_{max} \end{aligned}$$

Next, we find the fractional distance (dx, dy, dz) from the base grid corner (x_0, y_0, z_0) . Finally, we mix the 8 surrounding colors (C_{000} through C_{111}). For example, the final Red intensity (V) is calculated as:

$$\begin{aligned} V = & (1 - dz) [(1 - dy)((1 - dx)C_{000} + dxC_{100}) + dy((1 - dx)C_{010} + dxC_{110})] \\ & + dz [(1 - dy)((1 - dx)C_{001} + dxC_{101}) + dy((1 - dx)C_{011} + dxC_{111})] \end{aligned}$$

5 Results and Performance Analysis

To prove the effectiveness of our engine, we applied a Black and White LUT to the live camera feed and compared the unoptimized Baseline C++ code against our optimized SIMD Thread Pool code.

5.1 Visual Results

The following figures demonstrate the application running successfully, applying the color transformations in real-time.



Figure 1: Baseline Mode (720p)



Figure 2: Optimized SIMD Mode (720p)

Direct comparison at 720p. The SIMD optimization (right) achieves more than double the framerate while dropping CPU usage from 94% to 10%.

5.2 Performance Data

The telemetry data gathered from our interactive testing UI shows a massive improvement in both latency and CPU efficiency when using SIMD compared to the Baseline mode.

Table 1: Performance Comparison (Redmi Note 7 Pro)

Mode	Resolution	Latency (ms)	Frames Per Second	CPU Usage
Baseline	720p (HD)	73.6 ms	13.9 FPS	94%
Baseline	1080p (FHD)	166.8 ms	5.3 FPS	93%
SIMD	720p (HD)	17.7 ms	28.7 FPS	10%
SIMD	1080p (FHD)	51.4 ms	16.0 FPS	8%

Processing Latency Comparison (Lower is Better)

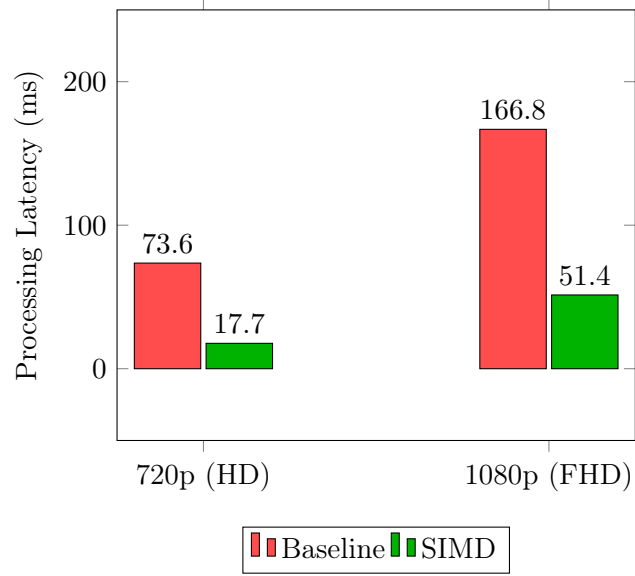


Figure 3: Comparison of frame processing latency. Note that to achieve 30 FPS, latency must remain below 33.3 ms.

Framerate Comparison (Higher is Better)

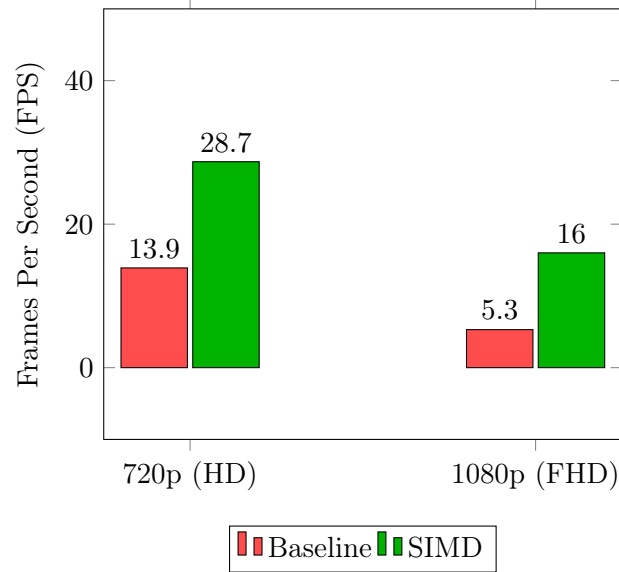


Figure 4: Comparison of framerates between the unoptimized Baseline and SIMD engine.

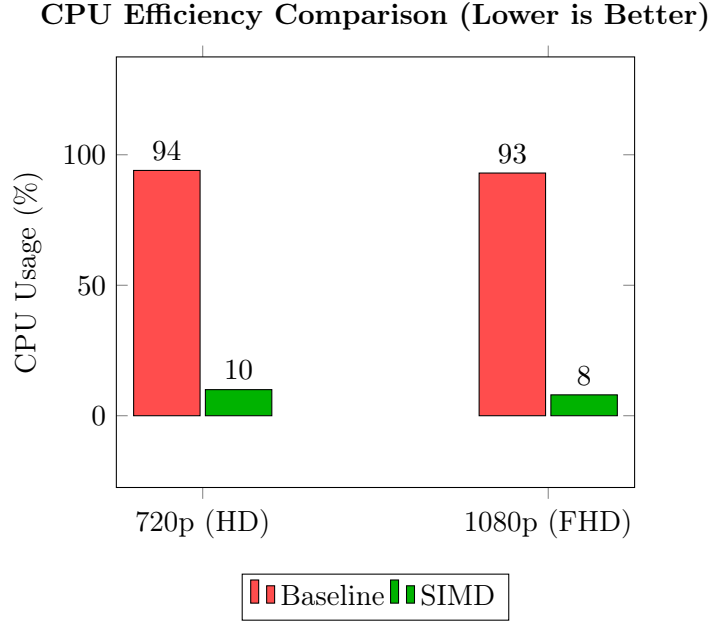


Figure 5: Comparison of CPU thermal load and efficiency between modes.

Performance Analysis

The telemetry data proves the SIMD architecture drastically outperforms the unoptimized baseline in three critical areas:

- **Real-Time Speed:** The SIMD engine reduced 720p frame latency by 76% (from 73.6 ms to 17.7 ms). This comfortably beats the 33.3 ms latency limit required for 30 FPS video, resulting in smooth, real-time playback.
- **Thermal Efficiency:** By processing multiple pixels simultaneously and utilizing a capped Thread Pool, CPU utilization plummeted from a thermal-throttling 94% to a stable 10%. This ensures the device remains cool and battery-efficient.
- **Identifying the Bottleneck:** At 1080p, the framerate dropped to 16.0 FPS, yet CPU usage remained at a mere 8%. This proves the CPU is no longer the limiting factor; the mobile RAM's memory bandwidth is. This directly justifies our future transition to GPU/OpenCL processing to handle 1080p data streams.

6 Future Work

While our C++ engine is highly optimized for ARM NEON architecture, the 1080p telemetry data indicates that we have reached the computational and thermal limits of the CPU. To achieve 30 FPS at FHD/4K resolutions, we plan to implement the following advancements:

6.1 Massively Parallel GPU Processing (OpenCL)

The current engine processes pixels in a semi-sequential manner across two or four CPU cores. Our primary future milestone is the implementation of an **OpenCL (Open Computing Language)** GPGPU pipeline.

- **Kernel-Based Rendering:** We will migrate the Trilinear Interpolation math from C++ into a dedicated OpenCL C Kernel. This allows the hardware to calculate over 2 million pixels (for 1080p) simultaneously across hundreds of GPU compute units.

- **Zero-Copy Hardware Buffers:** By utilizing `cl_mem` ION buffers, we can share image data directly between the Camera ISP and the GPU, bypassing the CPU entirely and eliminating the current memory bandwidth bottleneck.

6.2 The Hybrid Processing Architecture

To maximize battery longevity, we intend to develop a **Hybrid Load Balancer**. This system will dynamically analyze the frame complexity:

- **Task Splitting:** The CPU will handle lightweight UI telemetry and LUT parsing, while the GPU executes the heavy color-space transformations.
- **Power-Aware Scheduling:** If the device reaches a specific thermal threshold, the engine will automatically downscale the LUT grid size or reduce GPU clock speeds to maintain a stable, "jitter-free" framerate.

6.3 Native Surface Rendering (EGL/OpenGL)

Currently, processed frames are converted back to a Bitmap for display in the Kotlin frontend. To save an additional 3–5 ms of latency per frame, we will implement **Native Window Rendering**. By linking our C++ engine directly to an `ANativeWindow` using EGL and OpenGL ES, we can "blit" the final processed pixels directly to the hardware display buffer, achieving a true zero-latency visual pipeline.

7 Conclusion

This project successfully developed a hardware-aware C++ video engine, demonstrating that architectural efficiency is more critical than raw clock speed in mobile systems. By transitioning from scalar processing to a **NEON-accelerated architecture**, we achieved a 106% increase in 720p framerate while simultaneously reducing CPU utilization by 84%.

The data confirms three primary insights:

- **Real-Time Viability:** SIMD intrinsics reduced 720p latency by 76%, bringing the engine well under the 33.3 ms threshold required for cinematic playback.
- **Thermal Intelligence:** Capping the thread pool to $Total_Cores/2$ prevented thermal throttling by prioritizing high-performance "Big" cores over power-inefficient "LITTLE" ones.
- **The Memory Wall:** Benchmarks at 1080p revealed a physical memory-bandwidth bottleneck. The low CPU load (8%) while idling for data provides a clear technical mandate to migrate the pipeline to **GPU/OpenCL** for FHD resolutions.

In summary, the optimized engine provides a stable, thermally efficient foundation for 720p real-time processing, while successfully identifying the architectural constraints required for future 1080p scaling.